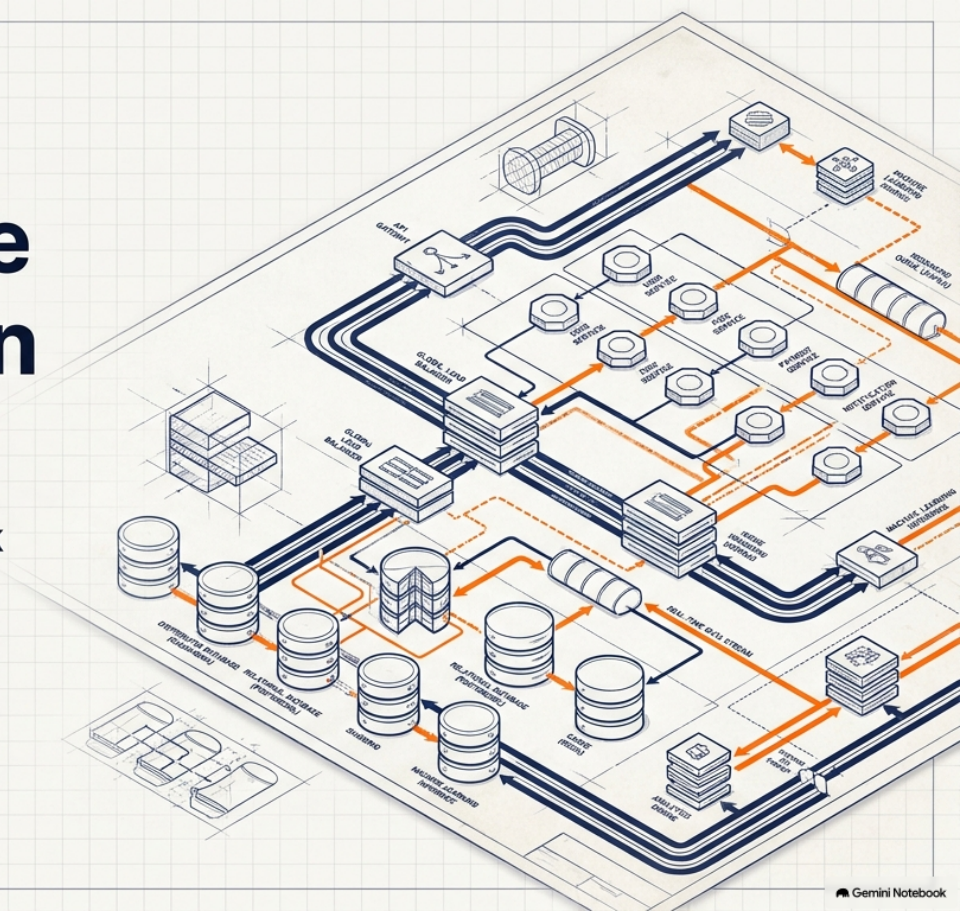


The 45-Minute System Design Blueprint

A Repeatable Mental Framework for Distributed Systems Architecture

CASE STUDY: UBER // ARCHITECTURE & SCALE



The Anatomy of a 45-Minute Interview

Premature Sharding
Irrelevant Cache Configs
Over-engineering the Load Balancer
Vague Error Handling
Inventing Math
Database Schema Anxiety
Micro-optimizing Locks
Forgotten Indexes
Unnecessary Queueing Systems
Protocol Obsession

05 MIN Requirements Defining functional limits and non-functional scope.	05 MIN Scale Estimation Deriving architecture from mathematical assumptions.
15 MIN High-Level Design The incremental build of core system components.	20 MIN Detailed Design Trade-offs, bottlenecks, and deep-dive problem solving.

Unnecessary Queueing Systems
Protocol Obsession
Assuming Infinite Resources
Vague Error Handling
Ignoring Consistency Models
Focus on Obscure Algorithms
Re-inventing Consensus
Neglecting Failure Modes

The goal is not to memorize designs. The goal is to demonstrate structure, estimation, prioritization, and the ability to defend architectural decisions.

Phase 1: Filter for Architectural Impact

Ask only the questions that materially affect the architecture.
Limit scope to 3-4 core functional requirements.

Low-Signal Questions	High-Signal Questions
Users: What color is the UI?	What is the read/write ratio?
Scale: Do we need an admin panel?	What is the expected latency and availability?
Geography: Do we support multiple languages?	Where are users located?



Geographic distribution fundamentally alters decisions regarding CDNs, replication, data residency, and regional architecture. **Always ask.**

Live Drill: Scoping the Uber System

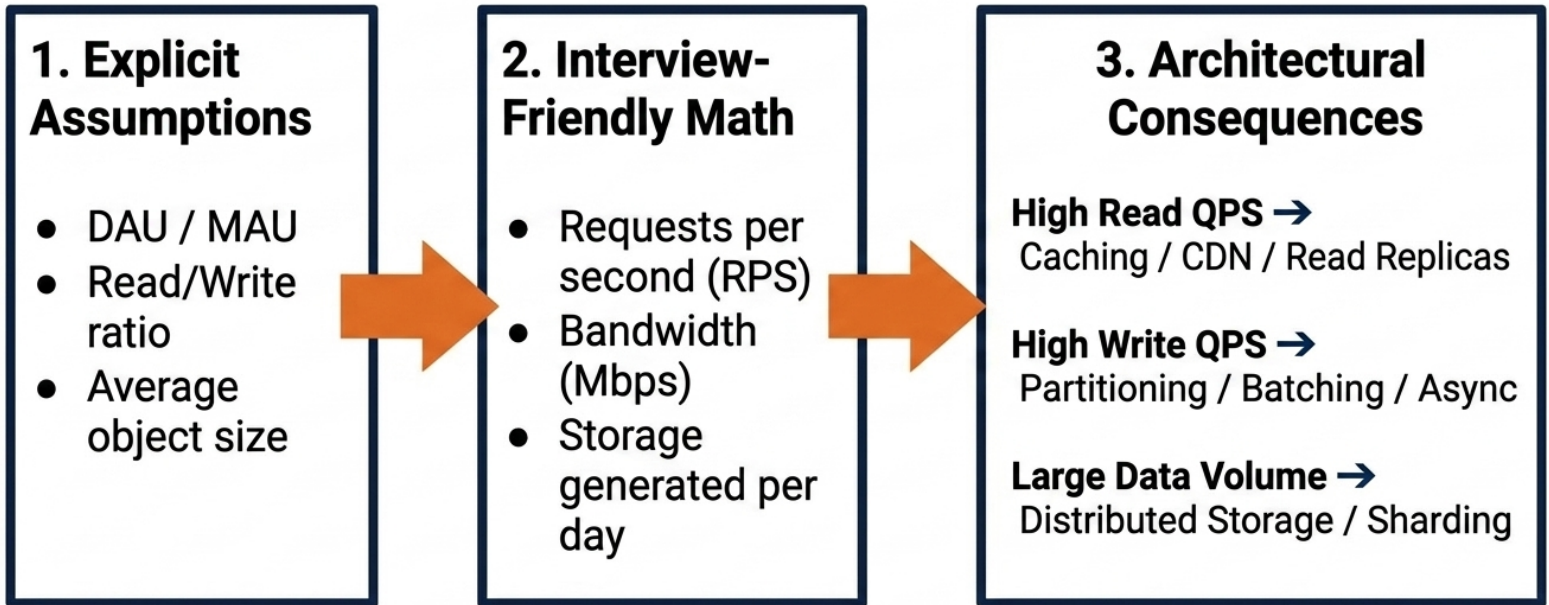
Functional Requirements

- **updateDriverLocation:** Track driver coordinates regularly.
- **findNearbyDrivers:** Display available drivers in vicinity.
- **requestRide:** Notify nearest driver of pickup.
- **managePayments:** Process start/end trip transactions.

Non-Functional Requirements

- **Availability:** Critical; downtime disrupts active trips.
- **Reliability:** Error-free requests and location updates.
- **Consistency:** Unified view of trip state for both parties.
- **Fraud Detection:** Identify malicious financial activity.

Phase 2: Math Dictates Architecture



ASSUMPTIONS → NUMBERS → ARCHITECTURAL CONSEQUENCES

Live Drill: Sizing the Ride-Hailing Network

The Assumptions Box

500M Riders (20M DAU) | **5M** Drivers (3M DAU) | **20M** Daily Trips



The Math

Bandwidth

20M trips / 86,400s $\approx$ **232 trips/sec**

3M active drivers * (3B ID + 16B loc) / 4s
 $\approx$ **114 Mbps**

Storage

Drivers: 108 MB/day

Trips: 2 GB/day

Total: ~2.71 GB/day

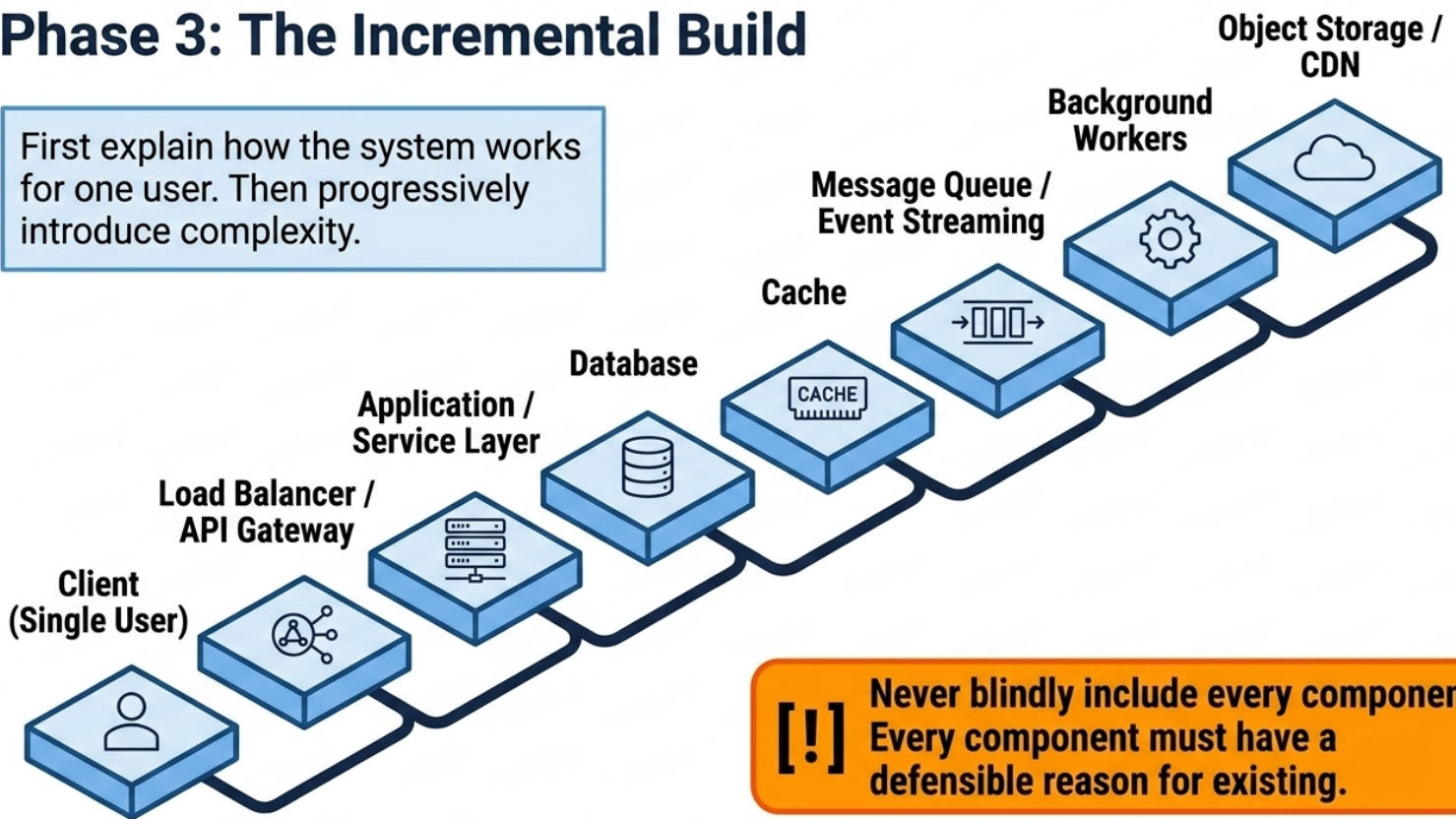
The Consequences

[!] 232 trips/sec average $\rightarrow$ Peak loads require burst handling.

[!] 114 Mbps bandwidth $\rightarrow$ Requires spatial partitioning (Quadtree) to optimize push updates.

Phase 3: The Incremental Build

First explain how the system works for one user. Then progressively introduce complexity.

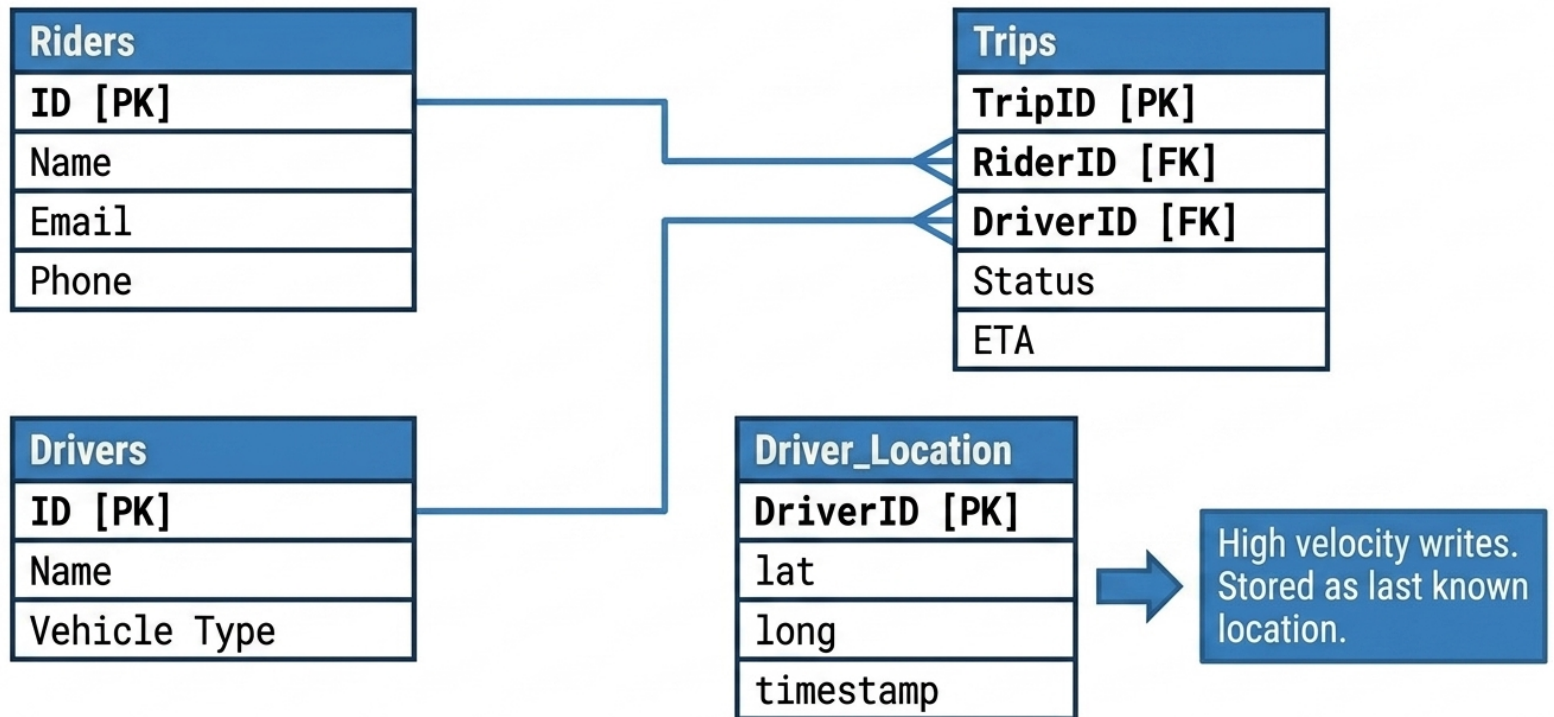


Never blindly include every component. Every component must have a defensible reason for existing.

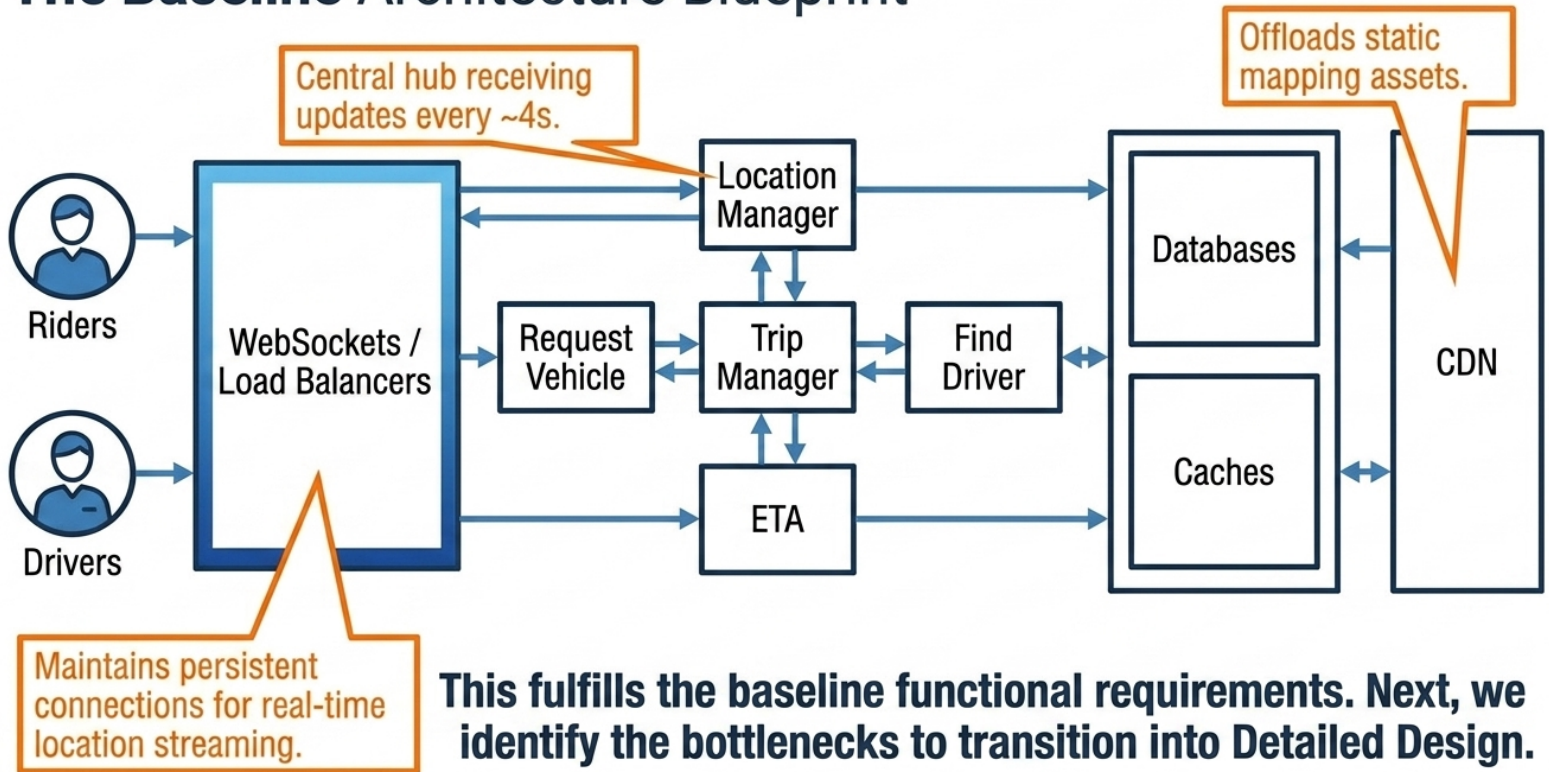
API Design: The 5 Core System Boundaries

Endpoint	Critical Payload	Function
updateDriverLocation()	driverID, oldlat, oldlong, newlat, newlong	Reconstruct route even with packet drops.
findNearbyDrivers()	riderID, lat, long	Identify available drivers in radius.
requestRide()	riderID, lat, long, dropOfflat, dropOfflong, type	Initiate ride request.
showETA()	rider_lat, rider_lng, dest_lat, dest_lng	Calculate driver arrival time.
showTripUpdates()	tripID, driverlat, driverlong, time_elapsed	Real-time status sync.

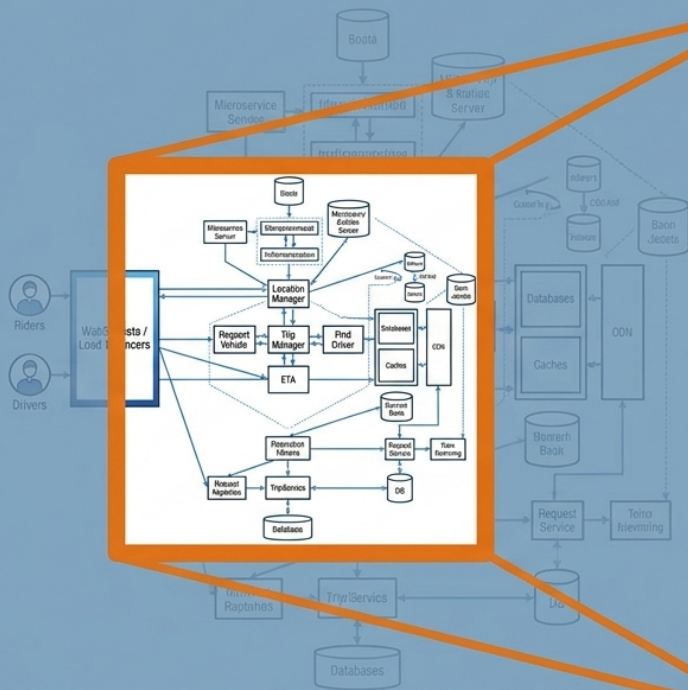
Data Model: The Storage Schema



The Baseline Architecture Blueprint



Phase 4: Go Deep Where It Matters

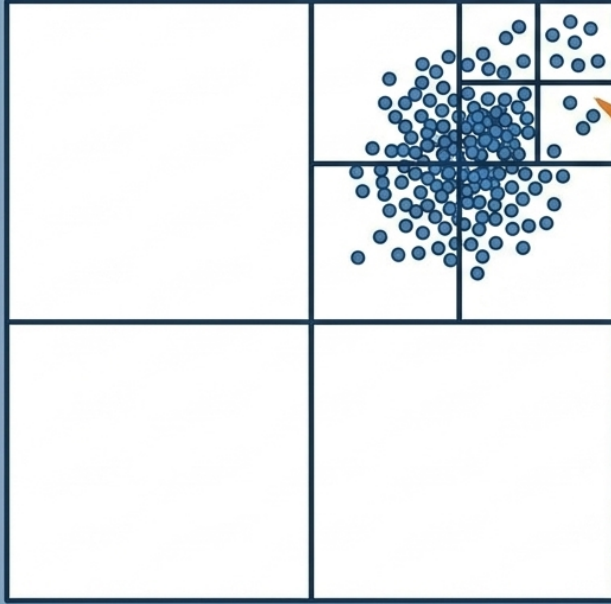


- Identify the most interesting technical challenge.
- Make and defend architectural decisions.
- Reason about trade-offs and distributed system failure scenarios.

For Uber, the baseline fails at scale. We must deep-dive into Geospatial Partitioning, ETA Routing, Database Selection, and Asynchronous Payments.

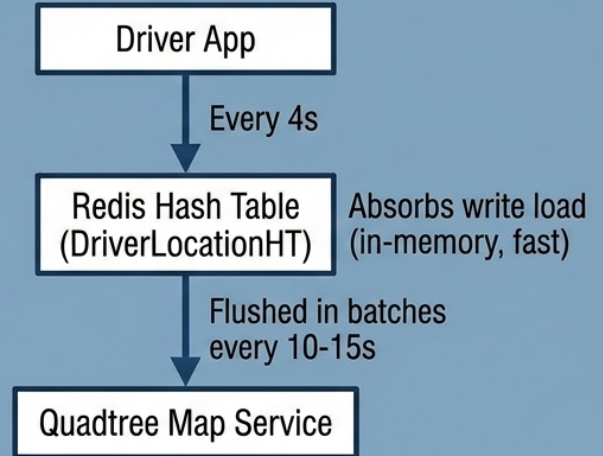
Bottleneck 1: The Write Velocity Problem

Quadtree Subdivision



Quadtree:
Re-calculating
regions constantly
is too expensive.

Dual-Layer Solution



The Trade-off: Absorbs massive write load while maintaining “good enough” geospatial accuracy.

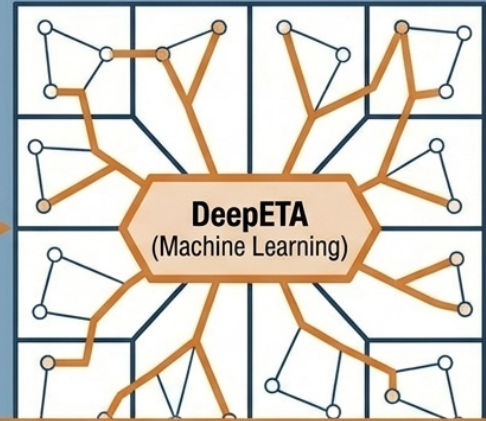
ETA Architecture: Beyond Dijkstra

The Limitation



Standard Dijkstra routing on an unprocessed graph is too slow for real-time ride-hailing latency.

The Distributed Solution



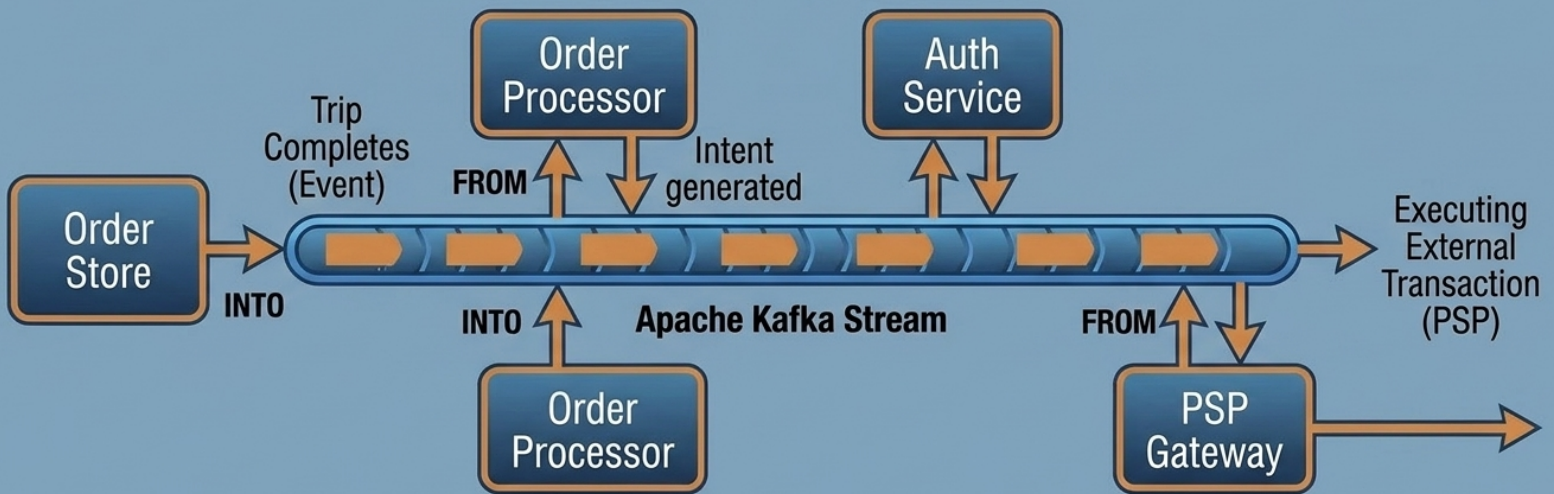
1. **Graph Partitioning:** Preprocess optimal paths within boundaries using contraction hierarchies.
2. **ML Component:** Forecast residual ETA using spatial, temporal, and real-time traffic data (edge weights).

Database Selection: Defending the Choice

	Cassandra (NoSQL)	MySQL (Relational)
Use Case	Driver last known location, completed trip history.	Active, in-progress trips.
Defending the Trade-off	Enormous data volume that never updates post-trip. High horizontal scalability and fault tolerance.	Requires strong consistency and ACID transactions across tables for live, updating state. Synchronous replication ensures reliability.

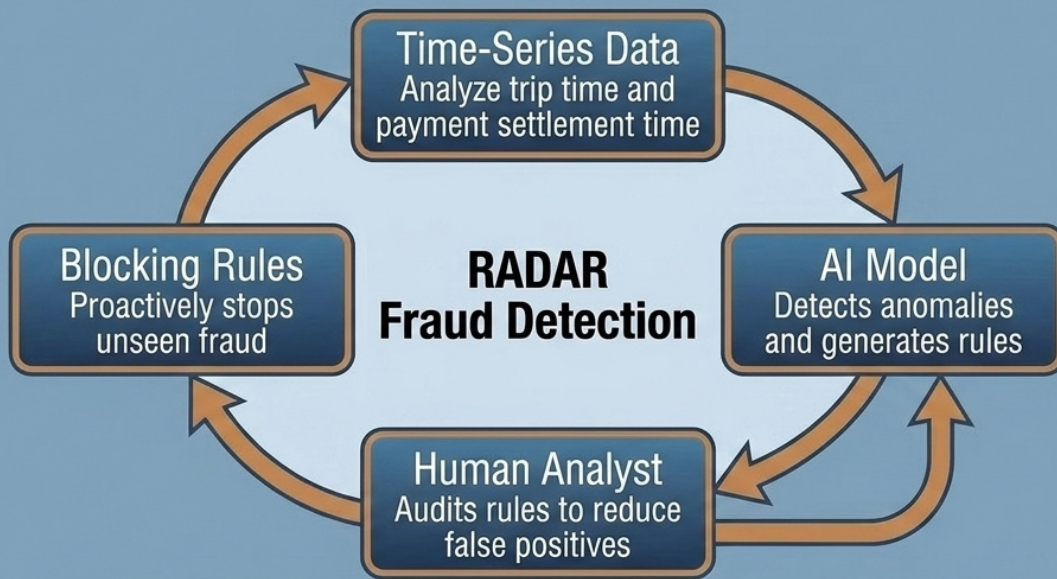
(Note: Uber later migrated to Google Cloud Spanner to merge global transactions with high scalability).

Asynchronous Payment Fault Tolerance



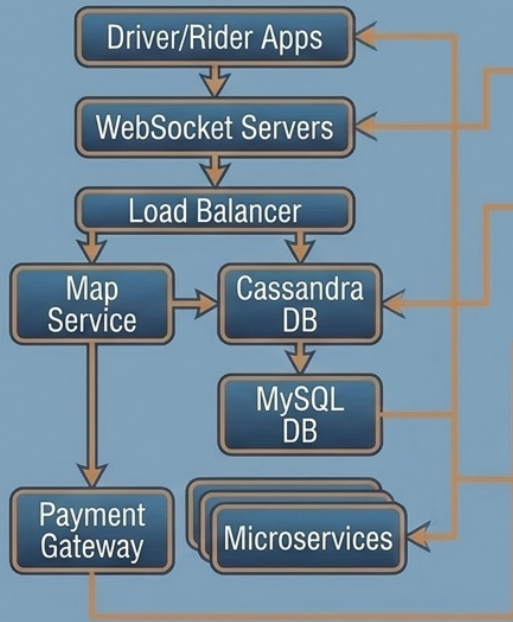
- Decouples internal trip completion from slow external Payment Service Providers (PSPs).
- Enables fault-tolerant record storage and Pub-Sub message queuing.
- Utilizes double-entry bookkeeping to prevent missing or duplicate payments.

System Integrity: Designing for Abuse



Context: Malicious actors spoof GPS and stall trips. A human-assisted AI loop ensures accountability at scale without compromising speed.

The Evaluation Matrix: Proving the Architecture



- [✓] **Availability:** Handled via WebSocket server replicas and Cassandra distributed architecture.
- [✓] **Scalability:** Horizontal sharding, Cassandra NoSQL, Quadtree spatial partitioning.
- [✓] **Reliability:** Phones act as local storage if connection drops; decoupled microservices.
- [✓] **Consistency:** Strong consistency for live trips using MySQL synchronous replication.

The Repeatable Mental Process

1. SCOPE

2. SIZE

3. BUILD

4. DEFEND

System design mastery is not architectural memorization. It is the disciplined execution of scoping, sizing, building incrementally, and defending trade-offs under pressure.

Assumptions dictate math. Math dictates bottlenecks. Bottlenecks dictate architecture. Own the process.